8. Sequential Composition

Function Composition

It's a familiar fact that functions can be "stacked together" or **composed**. For example, I could define *f* as:

$$f(x) = (x + 3) \times 2$$

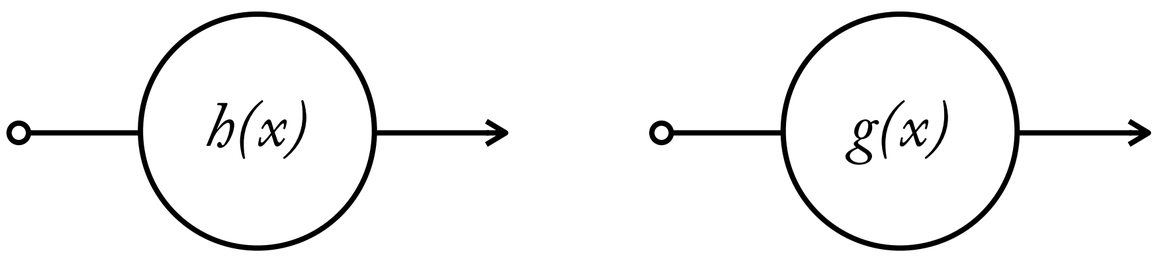
Here I've defined *f* in terms of a +3 function and ×2 function. Put another way, I've defined *f* as the composition of +3 and ×2.

Because functions are identified only by their arguments and values, the same function can be composed in different ways. In fact, "a composition" of a function is really just a set of instructions to the reader for how to identify that function. (It is very much like a computer program for your mind to run.) The same function can be specified using different sets of instructions. For example, consider the function *g* where:

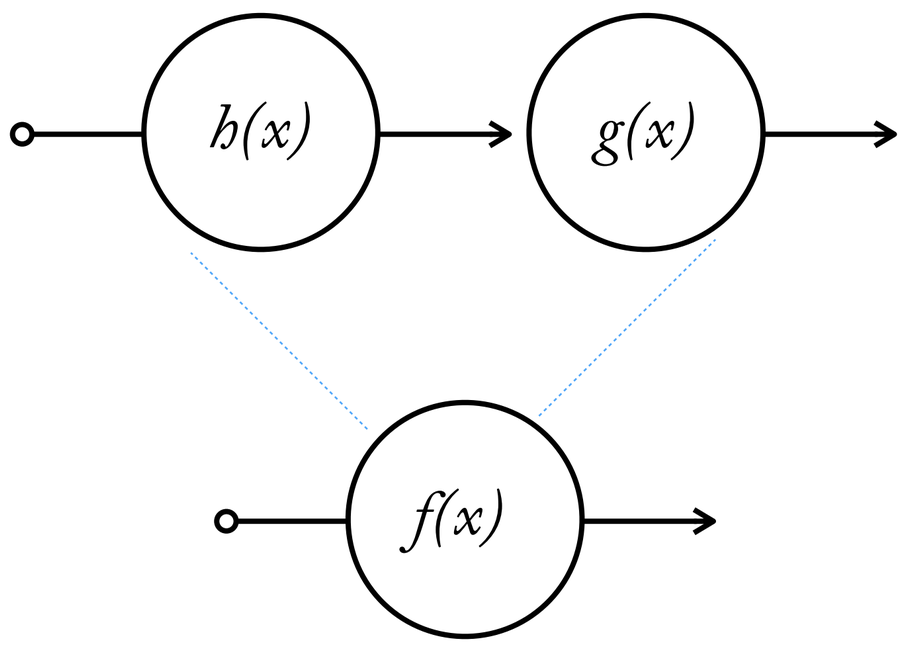
$$g(x) = x+7 = (x+5)+2$$

Here we describe *g* in two different ways, by composing different functions. Can you think of others we might have used?

Abstractly, given any two functions *h* and *g*:



You can define a third function *f* by composing together *h* and *g*:



$f(x) = g(h(x))$

Note that, in general, $g(h(x)) \neq h(g(x))$. So the order of composition matters.

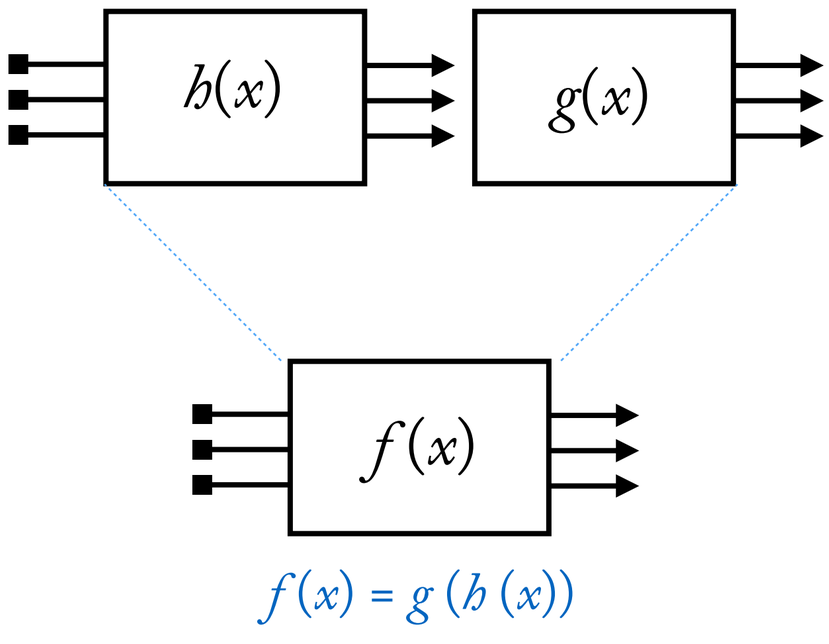
Machine Composition

Any time a function f can be defined in terms of the composition of functions g and h—then a *machine* which computes f can be designed by *putting together* a machine which computes g and a machine which computes h. (Actually, this is only true for certain ways of describing machines. But it holds for the circuit-based descriptions we're considering right now.)

For example, suppose: $f(x) = g(h(x))$. And suppose you have a CC that computes h and a CC that computes g:



Then you can construct a machine that computes f by *combining the machines themselves*. Like this:

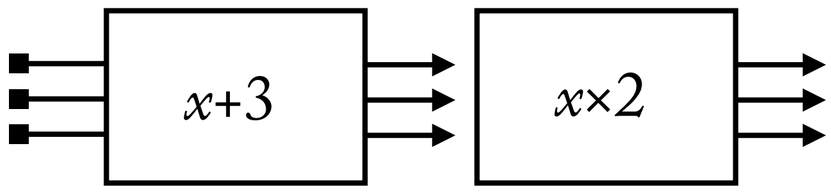


You'll have to be careful about the number of inputs and outputs lining up properly in each case. In the illustration here, I've abstracted away from this (important!) detail.

Concretely, applied to the example we started with, suppose:

$$f(x) = (x + 3) \times 2$$

And suppose you have machines that compute x + 3 and $x \times 2$. Then you can design a machine which computes f by putting together your $x + 3$ machine and your $x \times 2$ machine as follows:



You'll want to think about and experiment with these principles to make sure they work for both tally and binary, how to deal with multiple argument places, and other engineering questions that may arise.

Just as there are different ways of composing the same function, there are different ways of combining machines to compute the same function. If you have machines that compute +1, +2, and +5, can you describe different machines for computing $x + 7$?

Note! When composing functions, and putting machines together, *order matters*. (Recall the order of operations PEMDAS from middle school arithmetic!) For example, $2\times(3+x) \neq 3+(2\times x)$. This means that order also matters in the way you put together your machines to derive a more complex machine. I'll leave it to you to determine exactly *how* order matters here.